

even though there is considerable textual similarity across the text pair. It is also possible that the entailment system gets confused due to sub-classes being entailed across the two questions (namely, the clause, “Becker went on to become youngest winner”). This lends weight to our claim that while semantic similarity matching and textual entailment are closely related problems to the duplicate question detection task, they cannot be used as solutions directly for the duplicate detection problem.

There are subtle but important differences in the relations of entities—cross-sentence word level interaction between two sentences which mark them as non-duplicates when examined by humans. We can hypothesise that humans use these additional checks on top of the coarse grained similarity comparison they do in their minds when they look at these questions in isolation, and then arrive at the decision of whether they are duplicates or not. If we consider the example we discussed in Q2a and Q2b, the fact is that the relation between the entities in Question 2a does not hold good in Question 2b and, hence, if this cross-sentence level semantic relations are checked, it would be possible to determine that this pair is not a duplicate. It is also important to note that not all mismatches are equally important. Let us consider another example.

Example 3

Q3a: Do omega-3 fatty acids, normally available as fish oil supplements, help prevent cancer?

Q3b: Do omega-3 fatty acids help prevent cancer?

Though Q3b does not mention the fact that omega-3 fatty acids are typically available as fish oil supplements, its information needs are satisfied by the answer to Q3a, and hence these two questions are duplicates. From a human perspective, we hypothesise that the word fragment “normally available as fish oil supplements” is not seen as essential to the overall semantic compositional meaning of Q3a; so we can quickly discard this information when we refine the overall representation of the first question when doing a pass over the second question. Also, we can hypothesise that humans use cross-sentence word level interactions to quickly check whether similar information needs are being met in the two questions.

Example 4

Q4a: How old was Becker when he won the first time at Wimbledon?

Q4b: What was Becker’s age when he was crowned as the youngest winner at Wimbledon?

Though the surface forms of the two questions are quite dissimilar, humans tend to compare cross-sentence word level interactions such as (<old, age>, <won, crowned>) in the context of the entity in question, namely, Becker to conclude that these two questions are duplicates. Hence any system which attempts to solve the task of duplicate question detection should not depend blindly on a single aggregated coarse-grained similarity measure to compare the sentences,

but instead should consider the following:

- Do relations that exist in the first question hold true for the second question?
- Are there word level interactions across the two questions which cause them to have different informational needs (even if the rest of the question is pretty much identical across the two sentences)?

Now that we have a good idea of the requirements for a reasonable duplicate question detection system, let’s look at how we can start implementing this solution. For the sake of simplicity, let us assume that our data set consists of single sentence questions. Our system for duplicate detection first needs to create a representation for each input sentence and then feed the representations for each of the two questions to a classifier, which will decide whether they are duplicates or not, by comparing the representations. The high-level block diagram of such a system is shown in Figure 1.

First, we need to create an input representation for each question sentence. We have a number of choices for this module. As is common in most neural network based approaches, we use word embeddings to create a sentence representation. We can either use pre-trained word embeddings such as Word2Vec embeddings/Glove embeddings, or we can train our own word embeddings using the training data as our corpus. For each word in a sentence, we look up its corresponding word embedding vector and form the sentence matrix. Thus, each question (sentence) is represented by its sentence matrix (a matrix whose rows represent each word in the sentence and hence each row is the word-embedding vector for that word). We now need to convert the sentence-embedding matrix into a fixed length input representation vector.

One of the popular ways of representing an input sentence is by creating a sequence-to-sequence representation using recurrent neural networks. Given a sequence of input words (this constitutes the sentence), we now pass this sequence through a recurrent neural network (RNN) and create an output sequence. While RNN generates an output for each input in the sequence, we are only interested in the final aggregated representation of the input sequence. Hence, we take the output of the last unit of the RNN and use it as our sentence representation. We can use either vanilla RNNs, or gated recurrent units (GRU), or long short term memory (LSTM) units for creating a fixed length representation from a given input sequence. Given that LSTMs have been quite successfully used in many of the NLP tasks, we decided to use LSTMs to create the fixed length representation of the question.

The last stage output from each of the two LSTMs (one LSTM for each of the two questions) represents the input question representation. We then feed the two representations to a multi-layer perceptron (MLP) classifier. An MLP classifier is nothing but a fully connected multi-layer feed forward neural network. Given that we have